

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CURSO: DESARROLLO DE APLICACIONES MÓVILES



Ing. Walter Chamorro Palomino



Universidad
Tecnológica
del Perú

Sistema de Evaluación

Promedio Final:

$(10\%)APF1 + (20\%)APF2 + (30\%)APF3 + (40\%)PROY$

DESARROLLO DE APLICACIONES MÓVILES

Unidad 1

Introducción al desarrollo móvil

Logro de la Unidad 1:

Semanas 1, 2, 3, 4 y 5

Al finalizar la unidad, el estudiante desarrolla aplicaciones móviles básicas aplicando los fundamentos del desarrollo móvil.



Desarrollo de Aplicaciones Móviles

Manejo de eventos y estados en React Native

Componentes interactivos: Button, TouchableOpacity, TextInput

Semana 4



Universidad
Tecnológica
del Perú

Dudas sobre la sesión anterior



¿Qué trabajamos la sesión anterior?

- Principios de diseño móvil
- Componentes / fragments / props types
- Flexbox
- Creación de layouts responsivos con Flexbox.

Recordando los aprendizajes



¿Cuál es la función principal de los Props en React Native?

- A) Permitir la navegación entre pantallas
- B) Pasar datos de un componente padre a un componente hijo.
- C) Almacenar datos de forma persistente en la aplicación
- D) Estilizar los componentes con colores y tamaños.

Recordando los aprendizajes



¿Qué propiedad de Flexbox se usa para distribuir los elementos a lo largo del eje principal?

- A) `alignItems`
- B) `justifyContent`.
- C) `flexDirection`
- D) `alignSelf`.

Recordando los aprendizajes



En React Native, ¿cómo se aplican los estilos a un componente?

- A) Usando hojas de estilo CSS externas
- B) Con la propiedad style y un objeto JavaScript.
- C) Solo mediante librerías de terceros
- D) Con etiquetas `<style>` dentro del componente.

Recordando los aprendizajes



¿Cuál de las siguientes afirmaciones es correcta sobre Props, Fragments, Flexbox y Estilos en React Native?

- A) Los Props sirven para definir estilos de los componentes
- B) Los Fragments ayudan a agrupar elementos sin generar vistas adicionales.
- C) Flexbox solo funciona en aplicaciones web, no en móviles
- D) Los Estilos en React Native son idénticos a CSS tradicional.

Recordando los aprendizajes



¿Cuál es el propósito del símbolo `=>` en JavaScript?

- A) Es un operador para comparar valores entre dos variables
- B) Es una forma abreviada de declarar funciones (arrow function).
- C) Sirve para asignar valores a una variable
- D) Es un operador utilizado para crear bucles

Conocimientos previos



- ❖ ¿Qué entienden por "evento" dentro de una aplicación móvil?
- ❖ ¿Cuál creen que es la diferencia entre un evento y un estado en React Native?

Utilidad del tema



¿Por qué es importante conocer a los eventos, estados y componentes interactivos en React Native?

Utilidad:

- *Conocer eventos, estados y componentes interactivos es esencial porque permiten conectar la interacción del usuario con la actualización visual de la aplicación y constituyen la base para construir aplicaciones móviles dinámicas, usables y centradas en el usuario.*

Logro de la Semana 4

Al finalizar la semana 4, el estudiante comprende y aplica el manejo de eventos, estados y componentes interactivos en React Native, desarrollando interfaces móviles dinámicas que respondan adecuadamente a las acciones del usuario y mejoren la experiencia de interacción.



Contenido General

- ❖ Manejo de eventos y estados en React Native.
- ❖ Componentes interactivos: Button, TouchableOpacity, Pressable, TextInput.
- ❖ Implementación de un formulario básico con validaciones.



Componentes en React Native

Observemos el siguiente video y opinemos:



*¿Qué opinas de lo que
has visto en este video?*

Video Youtube:

<https://www.youtube.com/watch?v=9cYRdJPLuAk>

*Los participantes responden
de forma oral*

Manejo de eventos y estados en React Native

Manejo de eventos en React Native:

Los eventos permiten que la aplicación reaccione a las acciones del usuario (como toques, clics o escritura).

1. Eventos de Texto y Entrada de Datos:

- **onChangeText** → Para detectar cuando el usuario escribe en un TextInput.
- **onSubmitEditing** → Se dispara al presionar Enter o Aceptar en el teclado virtual.
- **onFocus / onBlur** → Detectan cuando un campo recibe o pierde el foco.

2. Eventos de Gestos y Toques

- **onPress** → Para detectar cuando un botón o un TouchableOpacity es presionado.
- **onLongPress** → Detecta una pulsación larga en un botón o área táctil.
- **onPressIn / onPressOut** → Se ejecutan cuando el usuario comienza o termina de presionar un elemento.

Manejo de eventos y estados en React Native

Manejo de eventos en React Native:

3. Eventos de Scroll y Listas

- **onScroll** → Para detectar el desplazamiento en un ScrollView o FlatList.
- **onEndReached** → Muy usado en FlatList, se dispara cuando el usuario llega al final de la lista (ideal para cargar más datos).

4. Eventos de Teclado y Pantalla

- **onKeyboardDidShow / onKeyboardDidHide** → Detectan cuando aparece o desaparece el teclado virtual.
- **onLayout** → Permite obtener las dimensiones y posición de un componente después de renderizarse.

Manejo de eventos y estados en React Native

Manejo de eventos en React Native:

Ejemplo simple:

```
import React from "react";
import { View, Text, Button, Alert } from "react-native";

export default function App() {
  const mostrarAlerta = () => {
    Alert.alert("¡Hola!", "Has presionado el botón.");
  };

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Ejemplo de Evento</Text>
      <Button title="Presióname" onPress={mostrarAlerta} />
    </View>
  );
}
```

Manejo de eventos y estados en React Native

Manejo de estados en React Native:

El estado (state) permite almacenar y actualizar información dentro de un componente.

Cuando el estado cambia, la interfaz se vuelve a renderizar automáticamente.

Se maneja con el Hook useState.

Ejemplo con useState:

```
import React, { useState } from "react";
import { View, Text, Button } from "react-native";

export default function App() {
  const [contador, setContador] = useState(0);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Contador: {contador}</Text>
      <Button title="Incrementar" onPress={() => setContador(contador + 1)} />
      <Button title="Reiniciar" onPress={() => setContador(0)} />
    </View>
  );
}
```

Preguntas



Componentes interactivos

1. Button

- Es el componente más sencillo para manejar acciones.
- Muestra un botón nativo (dependiendo de Android o iOS).
- Tiene la propiedad onPress para capturar eventos de clic.

Ejemplo:

```
import React from 'react';
import { Button, View, Alert } from 'react-native';

export default function App() {
  return (
    <View style={{ marginTop: 50 }}>
      <Button
        title="Presionar"
        onPress={() => Alert.alert("¡Botón presionado!")}
      />
    </View>
  );
}
```

Componentes interactivos

2. TouchableOpacity

- Más flexible que Button.
- Permite personalizar estilos (colores, bordes, imágenes, íconos, etc.).
- La opacidad baja automáticamente cuando se presiona.
- Ideal para botones personalizados.

Ejemplo:

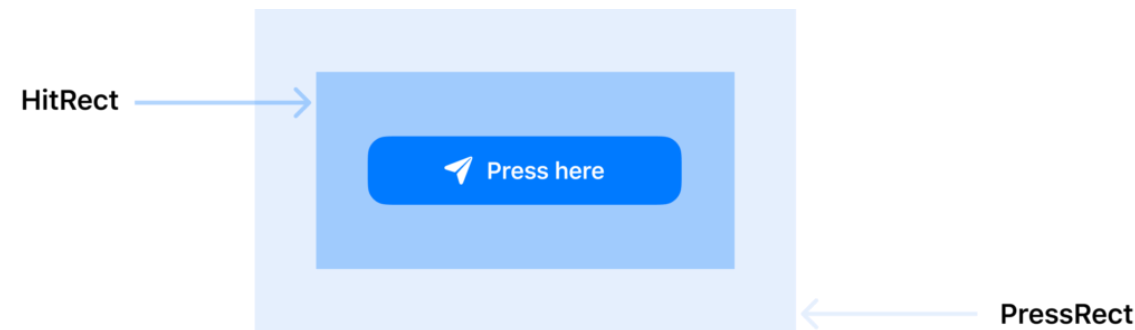
```
import React from 'react';
import { TouchableOpacity, Text, View, Alert } from 'react-native';

export default function App() {
  return (
    <View style={{ marginTop: 50, alignItems: "center" }}>
      <TouchableOpacity
        style={{
          backgroundColor: "blue",
          padding: 15,
          borderRadius: 10
        }}
        onPress={() => Alert.alert("¡Botón personalizado presionado!")}>
        <Text style={{ color: "white", fontSize: 18 }}>Clic Aquí</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Componentes interactivos

3. Pressable

- Permite aplicar diferentes estilos o ejecutar funciones dependiendo del estado de la interacción del usuario, por ejemplo:
 - ✓ Cuando el usuario presiona un botón.
 - ✓ Mientras el botón está siendo presionado.
 - ✓ Cuando se deja de presionar.
- Esto lo hace muy útil para crear botones personalizados e interfaces más dinámicas.



Componentes interactivos

3. Pressable

Ejemplo:

```
import React from 'react';
import { Pressable, Text, StyleSheet, View } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Pressable
        onPress={() => alert("¡Botón presionado!")}
        style={({ pressed }) => [
          {
            backgroundColor: pressed ? '#6C63FF' : '#2196F3'
          },
          styles.button
        ]}
      >
        <Text style={styles.text}>Presióname</Text>
      </Pressable>
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "#f5f5f5",
  },
  button: {
    paddingVertical: 12,
    paddingHorizontal: 20,
    borderRadius: 8,
  },
  text: {
    color: "white",
    fontSize: 16,
    fontWeight: "bold",
  },
});
```

Componentes interactivos

Comparación:

◆ Button

- **Uso principal:** Botón simple, rápido de implementar.
- **Ventajas:**
 - Muy fácil de usar.
 - Viene con estilo predeterminado.
- **Limitaciones:**
 - Personalización limitada en estilos.
 - No permite manejar diferentes estados (ej: presionado, enfocado).

◆ Pressable

- **Uso principal:** Crear componentes **altamente personalizables e interactivos**.
- **Ventajas:**
 - Control total de estilos en diferentes estados:
 - `pressed` (cuando está presionado).
 - `hovered` (cuando se pasa el cursor en web).
 - `focused` (cuando recibe foco con teclado).
 - Ideal para botones complejos y accesibles.
- **Limitaciones:**
 - Requiere más código que `Button`.

◆ TouchableOpacity

- **Uso principal:** Hacer que cualquier vista sea "tocable" con un **efecto de opacidad** al presionar.
- **Ventajas:**
 - Se puede personalizar el contenido interno (texto, íconos, imágenes).
 - Da feedback visual con el cambio de opacidad.
- **Limitaciones:**
 - Menos control sobre estados avanzados comparado con `Pressable`.

Componentes interactivos

4. TextInput

- Se usa para entradas de texto (nombre, correo, contraseña, etc.).
- Tiene propiedades como placeholder, secureTextEntry (para contraseñas) y eventos como onChangeText.

Ejemplo:

```
import React, { useState } from 'react';
import { TextInput, View, Text } from 'react-native';

export default function App() {
  const [texto, setTexto] = useState("");

  return (
    <View style={{ marginTop: 50, padding: 20 }}>
      <TextInput
        style={{
          borderWidth: 1,
          borderColor: "gray",
          padding: 10,
          borderRadius: 5
        }}
        placeholder="Escribe aquí..."
        onChangeText={nuevoTexto => setTexto(nuevoTexto)}
      />
      <Text style={{ marginTop: 20, fontSize: 18 }}>
        Has escrito: {texto}
      </Text>
    </View>
  );
}
```

Preguntas



Desarrollo de Aplicaciones Móviles



Desarrollo de Aplicaciones Móviles

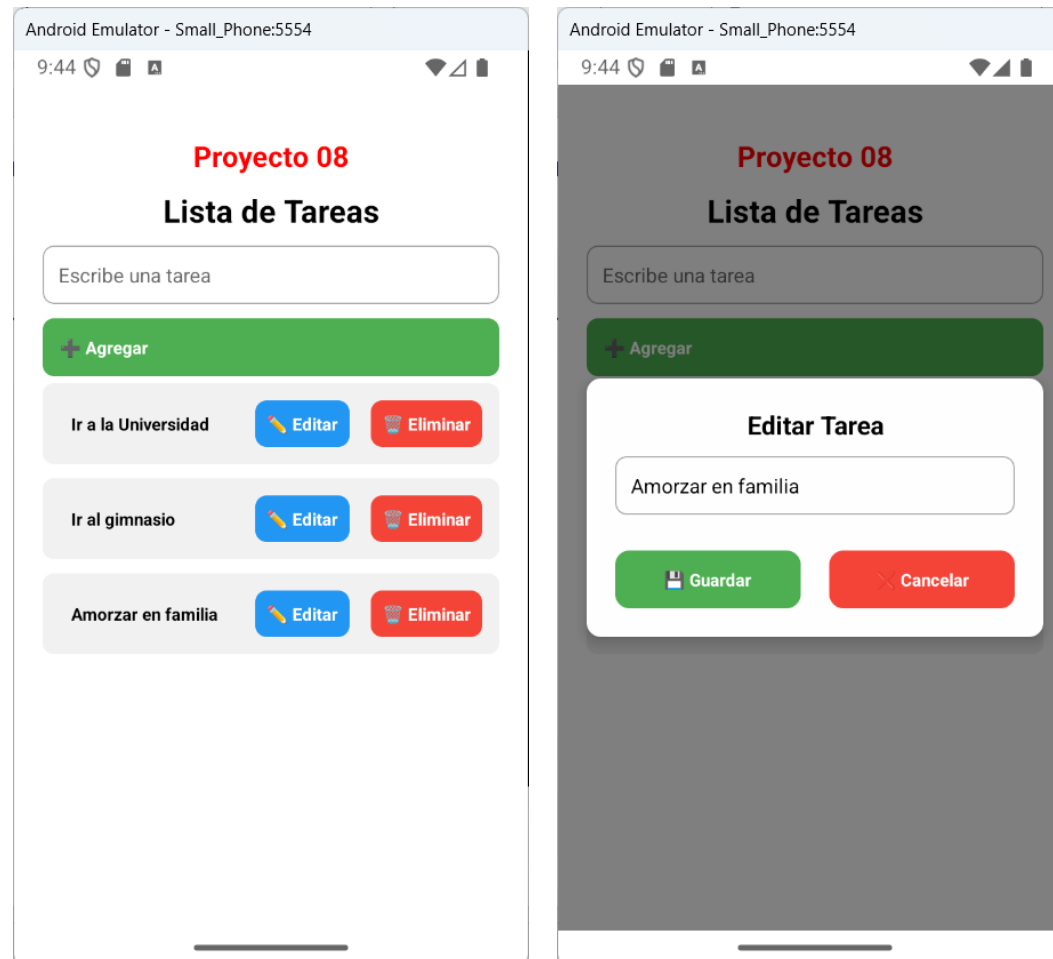
Práctica



Práctica

Ejercicio:

Desarrolle una aplicación móvil en React Native para gestionar Tareas (listar, agregar, modificar y eliminar), la interfaz debe mostrar como la siguiente imagen:



Preguntas



Principios de diseño móvil

Actividad grupal:

En grupos de hasta 8 estudiantes diseñe una aplicación móvil con contenido libre en donde se evidencie todos los temas tratados en esta semana.



Resumen de la Clase

- ✓ **Eventos:** En React Native, los eventos permiten detectar e interactuar con las acciones del usuario, como toques, clics, desplazamientos o entradas de texto. Se manejan a través de funciones (event handlers) que responden cuando ocurre una acción, por ejemplo: `onPress`, `onChangeText`, `onScroll`, entre otros.
- ✓ **Estados:** El estado (state) es un objeto que representa la información dinámica de un componente. Permite que la interfaz se actualice automáticamente cuando los datos cambian. Se gestiona mediante hooks como `useState`, facilitando la creación de aplicaciones reactivas y actualizadas en tiempo real.
- ✓ **Componentes interactivos:** React Native incluye componentes listos para capturar la interacción del usuario:
 - **Button:** Botón simple con acción rápida al presionar.
 - **TouchableOpacity:** Similar a un botón, pero con efectos de opacidad y mayor flexibilidad de diseño.
 - **Pressable:** Más avanzado, permite controlar diferentes estados de interacción (presionado, liberado, mantenido).
 - **TextInput:** Campo para introducir y controlar texto escrito por el usuario.



Pregunta de cierre



- ¿Cómo se relacionan los eventos con el estado en una aplicación de React Native para lograr interactividad?
- ¿Qué ventajas ofrece el uso de componentes interactivos como Button, Pressable o TouchableOpacity en el diseño de interfaces móviles?

Conclusiones

- Los eventos son la base de la interacción en React Native, ya que permiten que la aplicación responda a las acciones del usuario en tiempo real.
- El estado es clave para gestionar la información dinámica, asegurando que la interfaz se actualice automáticamente cuando los datos cambian.
- Los componentes interactivos (como Button, TouchableOpacity y Pressable) facilitan la construcción de interfaces más atractivas y funcionales.
- La combinación de eventos, estados y componentes garantiza aplicaciones móviles dinámicas, reactivas y centradas en la experiencia del usuario.
- Comprender estos conceptos fortalece las bases del desarrollo en React Native y es esencial para construir aplicaciones modernas e intuitivas.



Gracias

